

EECS 482

Lab 01: Threads and Race Conditions

Administrivia

Today: 5/16

Due dates

Group declarations

5/28

Project 1

6/02

Agenda

- Processes & threads
- Race conditions & synchronization
- Lab exercises
- Project 1

— — —

Week in review

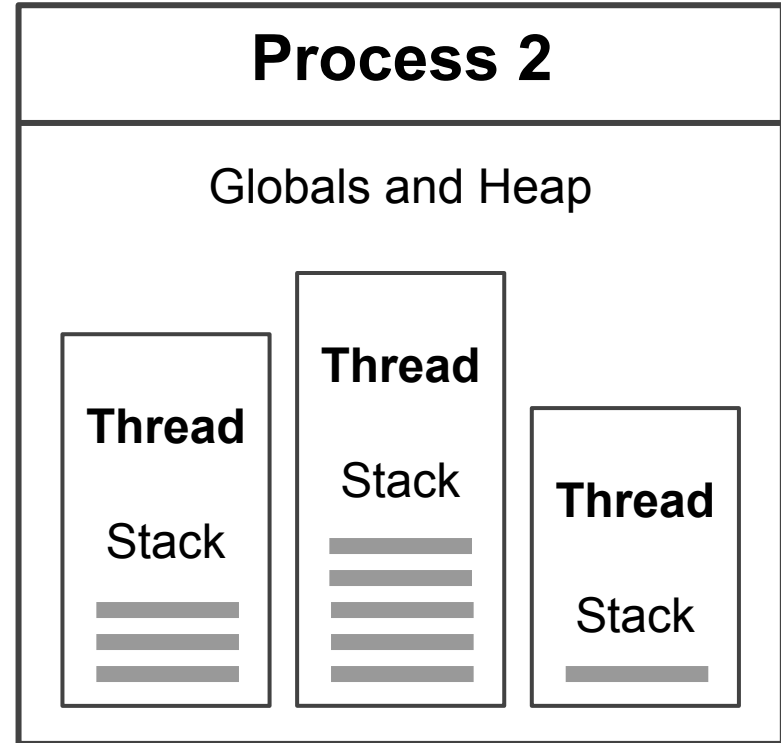
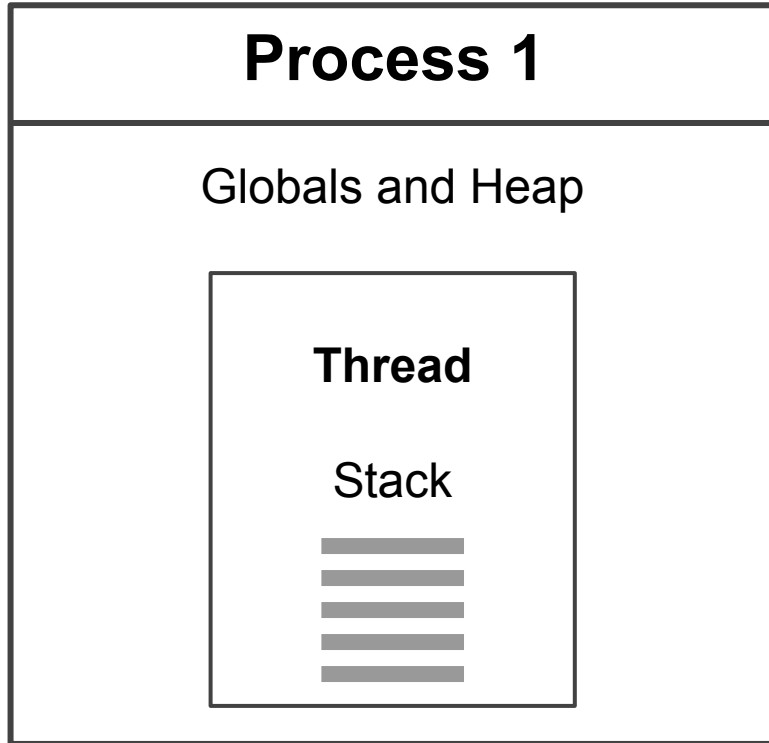
— — —

- Processes, threads, and shared memory
- Race conditions
 - Too Much Milk
- Atomic operations
- Synchronization
 - Critical sections
 - Mutual exclusion (mutex)

Processes, threads, and shared memory

How do threads relate to processes?

Threads within a process share memory



Why do we want threads?

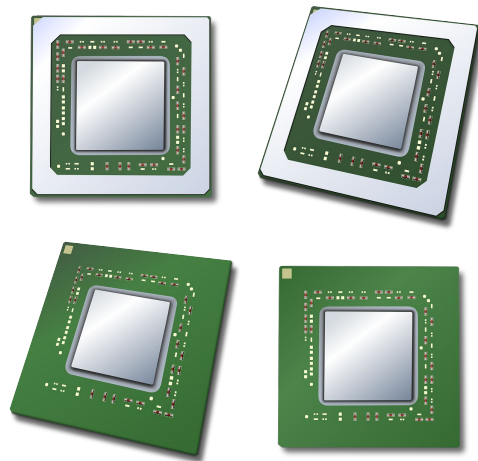


Why do we want threads?

Make use of CPU while
waiting on **slow I/O**



Utilize multiple CPUs to
parallelize computation



Atomicity, race conditions, and synchronization

Atomic operations are indivisible.

Thread 1

```
code();  
more_code();
```

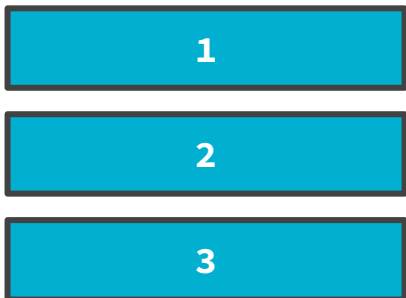
Thread 2

```
other_code();  
codie_code();  
hello_world();
```

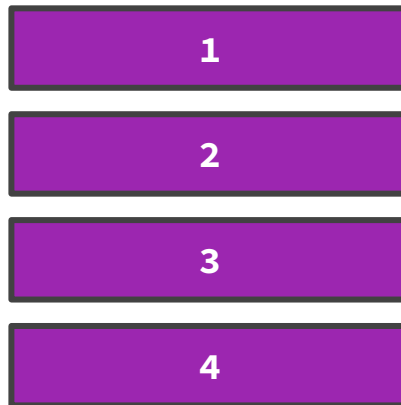
*This is a simplification, as functions are probably not atomic

Atomic operations are indivisible.

Thread 1

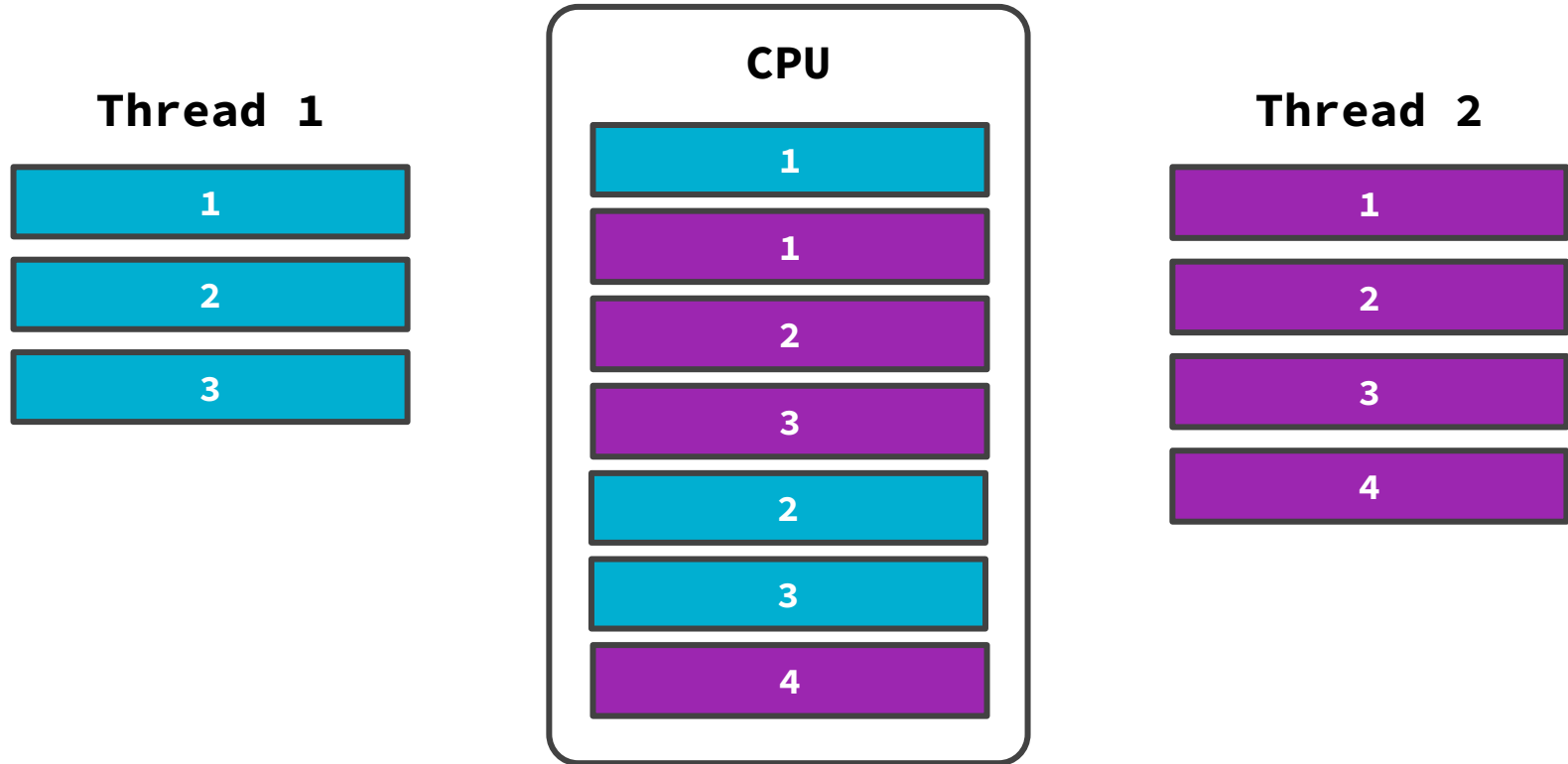


Thread 2



*This is a simplification, as functions are probably not atomic

Atomic operations are indivisible.



Don't assume any code you write is atomic!

...unless it is explicitly guaranteed.

Is this atomic? (32 bit system)

```
int32_t x;
```

```
...
```

```
x++;
```

Probably not.

```
temp = x + 1;  
x = temp;
```

Is this atomic? (32 bit system)

```
int32_t x;
```

```
...
```

```
x = 1;
```

Probably. It is likely 1 atomic memory store.

Is this atomic? (32 bit system)

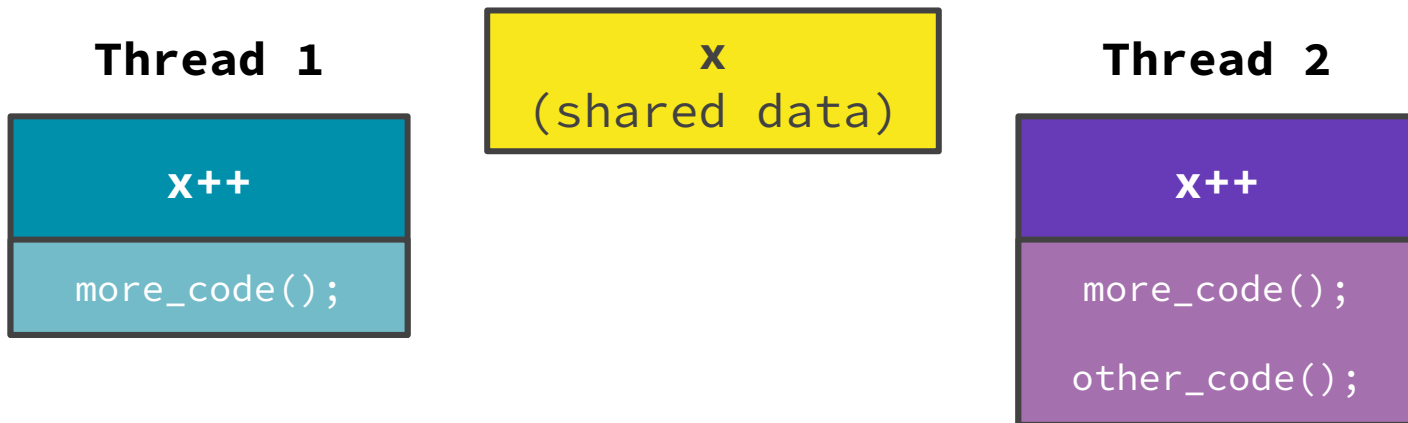
```
int64_t x;
```

```
...
```

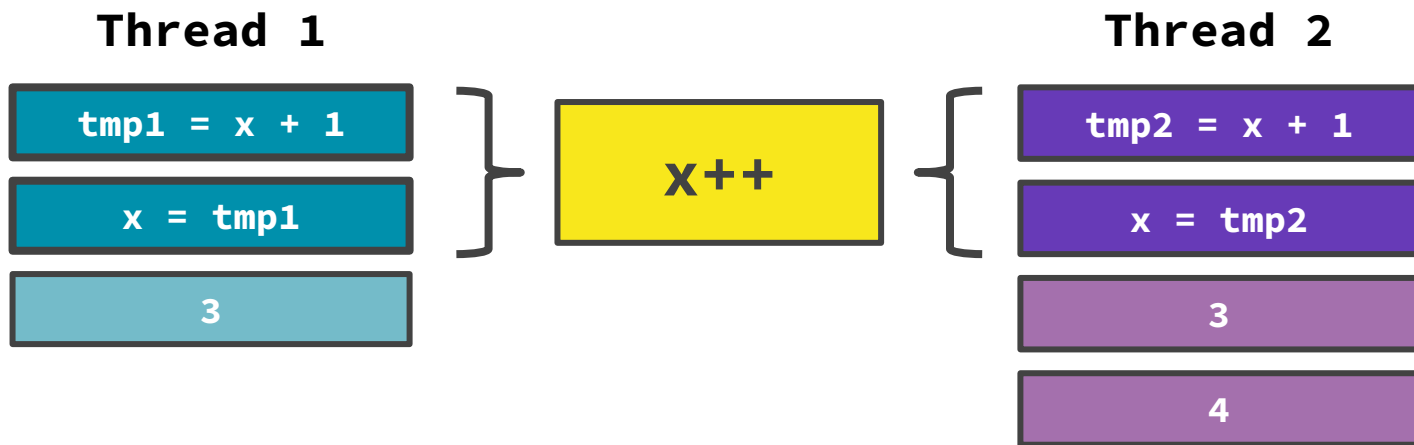
```
x = 1;
```

Probably not. It is most likely 2 memory stores, one to the lower 32 bits and one for the upper 32 bits.

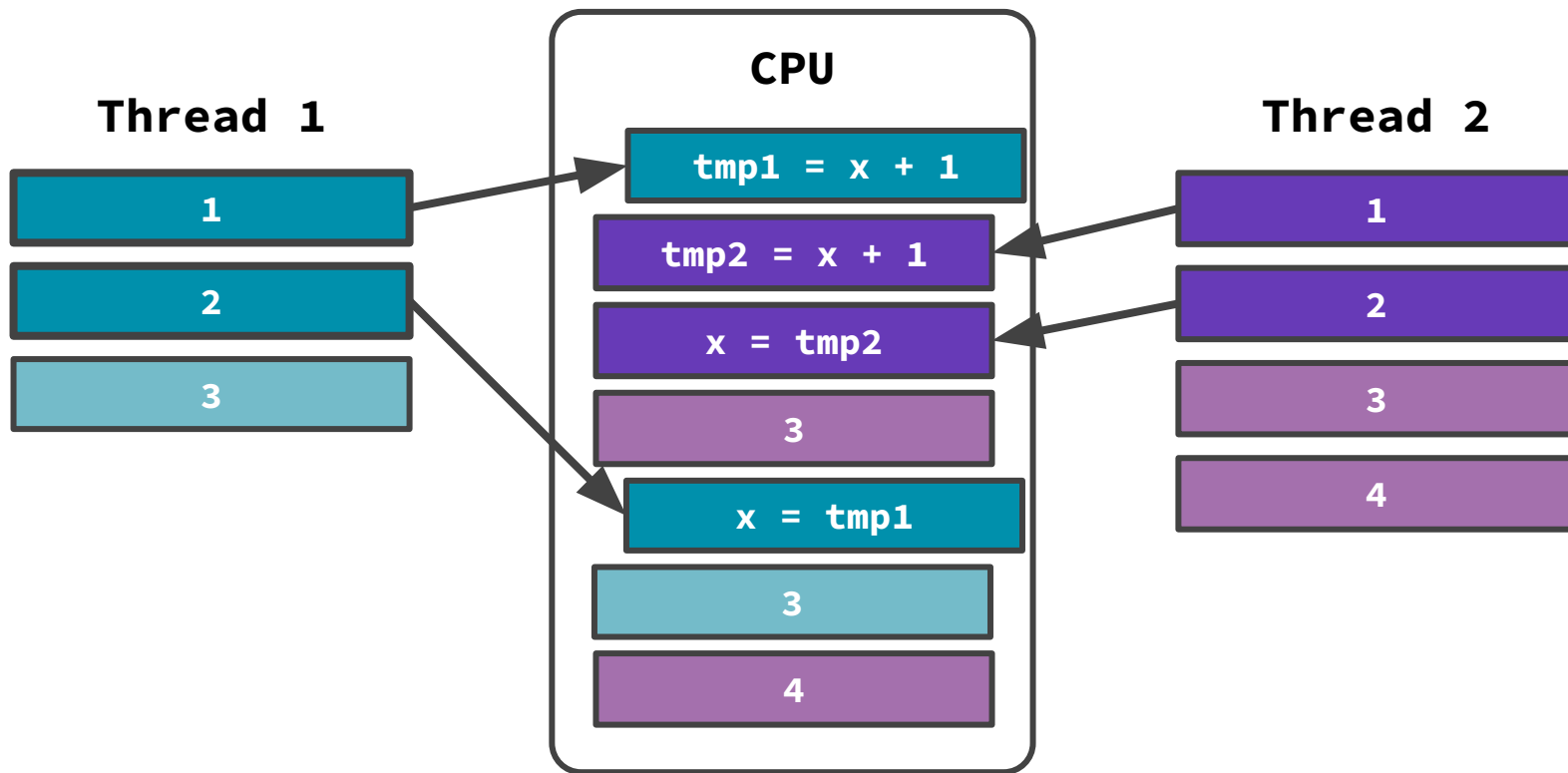
Operations on shared data form **critical sections**.



Operations on shared data form **critical sections**.



Non-atomic critical sections lead to **race conditions**.



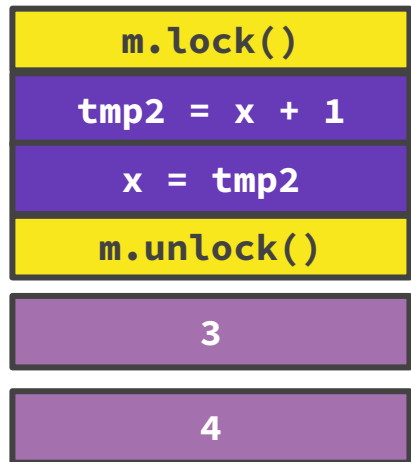
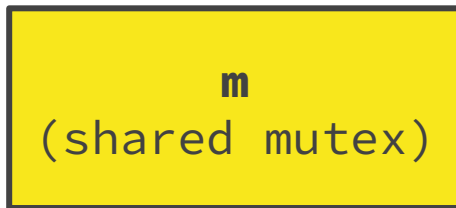
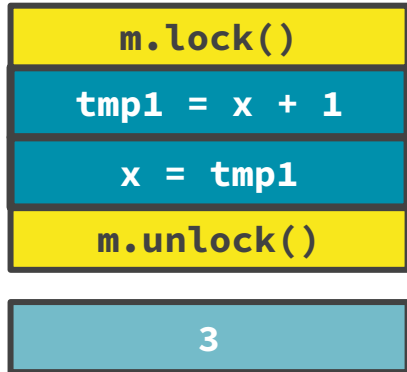
Synchronization prevents race conditions.

We need to manage how multiple threads are allowed to access certain things.

1. **Mutual exclusion:** only one thread can do this at a time (mutex)
2. **Ordering:** this thread must wait until some condition is true (condition variable)

Mutexes provide mutual exclusion between critical sections

- **lock()**: acquire mutex or waits until released
- **unlock()**: release ownership of the mutex



Condition variables provide communication between threads

- **wait(mutex &m):** wait on the cv
 - m.unlock();
 - go on a waiting queue
 - sleep until signal/broadcast is called
 - m.lock();
- **signal():** wake one thread waiting on the cv
- **broadcast():** wake all threads waiting on the cv

Exercise 1: Threads and races

Exercise 1: Threads and races

```
const int NUM_THREADS = 2;  
const int NUM_ITER = 20;
```

```
int counter = 0;
```

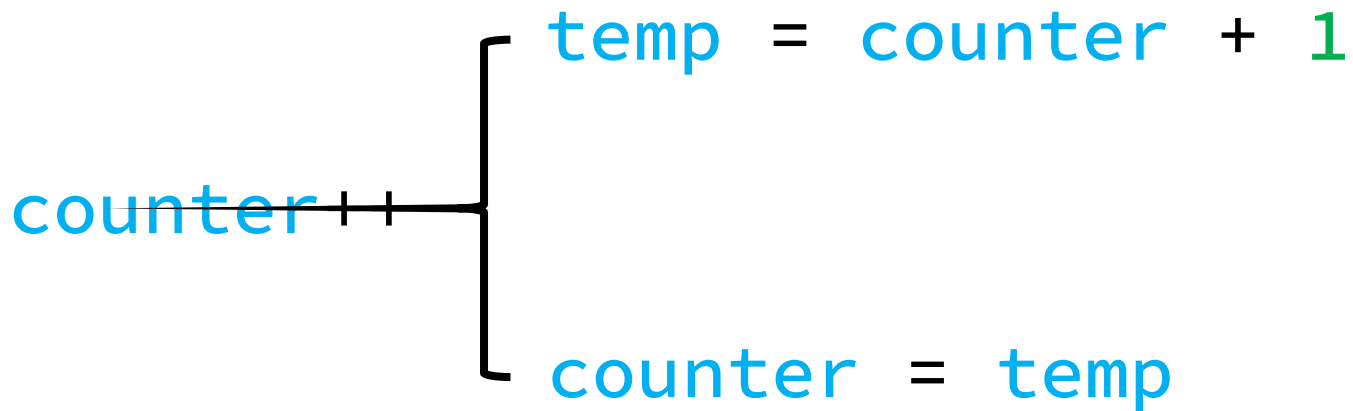
```
void child() {  
    for (int i = 0; i < NUM_ITER; i++) {  
        counter++;  
    }  
}
```

What is maximum value
of counter?

```
void main_thread() {  
    for (int i = 0; i < NUM_THREADS; i++) {  
        thread t(child);  
    }  
}
```

Exercise 1: Threads and races

Assume that assignment and addition (but **not** increment) are atomic.



Exercise 1A

```
const int NUM_THREADS = 2;  
const int NUM_ITER = 20;
```

```
int counter = 0;
```

```
void child() {  
    for (int i = 0; i < NUM_ITER; i++) {  
        counter++;  
    }  
}
```

What is the **maximum value** that counter can have afterward?

Exercise 1A: max value = 40

— — —

counter = 0

Thread 0

Thread 1

Exercise 1A: max value = 40

— — —

counter = 20

Thread 0

counter = counter (0) + 1

...

counter = counter (19) + 1

Thread 1

Exercise 1A: max value = 40

— — —

counter = 40

Thread 0

counter = counter (0) + 1

...

counter = counter (19) + 1



Thread 1

counter = counter (20) + 1

...

counter = counter (39) + 1

Exercise 1A

What is the maximum possible value of counter at the end of the program?

40

Are there any other interleavings that lead to this maximum value?

Exercise 1A: another example of 40

— — —

counter = 0

Thread 0

Thread 1

Exercise 1A: another example of 40

— — —

counter = 1

Thread 0

Thread 1

counter = counter (0) + 1

Exercise 1A: another example of 40

— — —

counter = 2

Thread 0

counter = counter (0) + 1

Thread 1

counter = counter (1) + 1

Exercise 1A: another example of 40

— — —

counter = 3

Thread 0

counter = counter (0) + 1

counter = counter (2) + 1

Thread 1

counter = counter (1) + 1

Exercise 1A: another example of 40

— — —

counter = 40

Thread 0

counter = counter (0) + 1

counter = counter (2) + 1

...

Thread 1

counter = counter (1) + 1

...

Exercise 1A

```
const int NUM_THREADS = 2;  
const int NUM_ITER = 20;
```

```
int counter = 0;
```

```
void child() {  
    for (int i = 0; i < NUM_ITER; i++) {  
        counter++;  
    }  
}
```

What is the **minimum value** that counter can have afterward?

Exercise 1A: min value = 2

— — —

counter = 0

Thread 0

Thread 1

Exercise 1A: min value = 2

— — —

counter = 0

Thread 0

Thread 1

temp = counter (0) + 1 //1

Exercise 1A: min value = 2

counter = 1

Thread 0

temp = counter (0) + 1 //1

Thread 1

counter = counter (0) + 1

Exercise 1A: min value = 2

counter = 19

Thread 0

```
temp = counter (0) + 1 //1
```

Thread 1

```
counter = counter (0) + 1
```

...

```
counter = counter (18) + 1
```

Exercise 1A: min value = 2

counter = 1

Thread 0

```
temp = counter (0) + 1 //1
```

```
counter = temp
```

Thread 1

```
counter = counter (0) + 1
```

...

```
counter = counter (18) + 1
```

Exercise 1A: min value = 2

counter = 1

Thread 0

```
temp = counter (0) + 1 //1
```

```
counter = temp
```

Thread 1

```
counter = counter (0) + 1
```

```
...  
counter = counter (18) + 1
```

```
temp = counter (1) + 1 //2
```

Exercise 1A: min value = 2

counter = 20

Thread 0

```
temp = counter (0) + 1 //1
```

```
counter = temp
```

```
counter = counter (1) + 1
```

...

```
counter = counter (19) + 1
```

Thread 1

```
counter = counter (0) + 1
```

...

```
counter = counter (18) + 1
```

```
temp = counter (1) + 1 //2
```

Exercise 1A: min value = 2

counter = 2

Thread 0

```
temp = counter (0) + 1 //1
```

```
counter = temp
```

```
counter = counter (1) + 1
```

...

```
counter = counter (19) + 1
```

Thread 1

```
counter = counter (0) + 1
```

...

```
counter = counter (18) + 1
```

```
temp = counter (1) + 1 //2
```

```
counter = temp
```

Exercise 1B

On CAEN, compile the program, then use a scripting language to run it 10000 times and summarize the results.

- `g++ -std=c++20 race.cpp libthread.o -o race -ldl -lpthread`

`./race`

`(NUM_THREADS = 2; NUM_ITER = 20)`

Link at: <https://os.eecs.umich.edu/lab/threads.html>

Exercise 1B: Sample Results

```
bait@caen-vnc-vm08:~/ia/bait.lab/lab01$ python3 run.py
```

```
How many iterations? 100000
```

```
Results:
```

```
counter = 40 occurred 99716 times
```

```
counter = 20 occurred 245 times
```

```
counter = 37 occurred 7 times
```

```
counter = 24 occurred 1 times
```

```
counter = 34 occurred 3 times
```

```
counter = 31 occurred 1 times
```

```
counter = 38 occurred 6 times
```

```
counter = 30 occurred 2 times
```

```
counter = 39 occurred 8 times
```

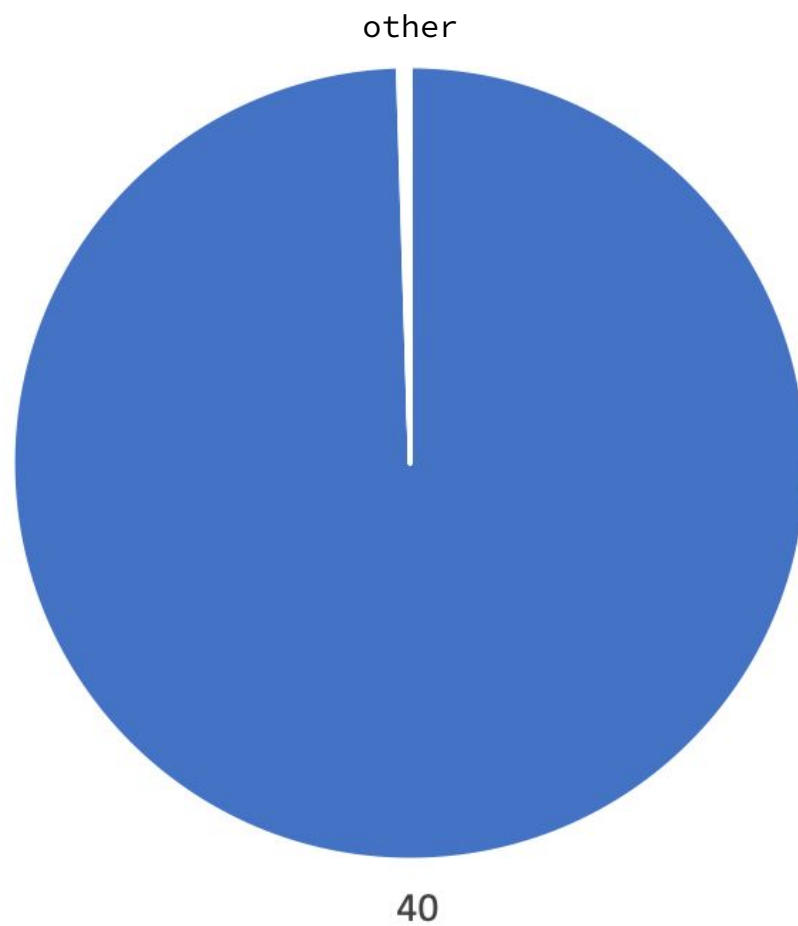
```
counter = 33 occurred 3 times
```

```
counter = 36 occurred 5 times
```

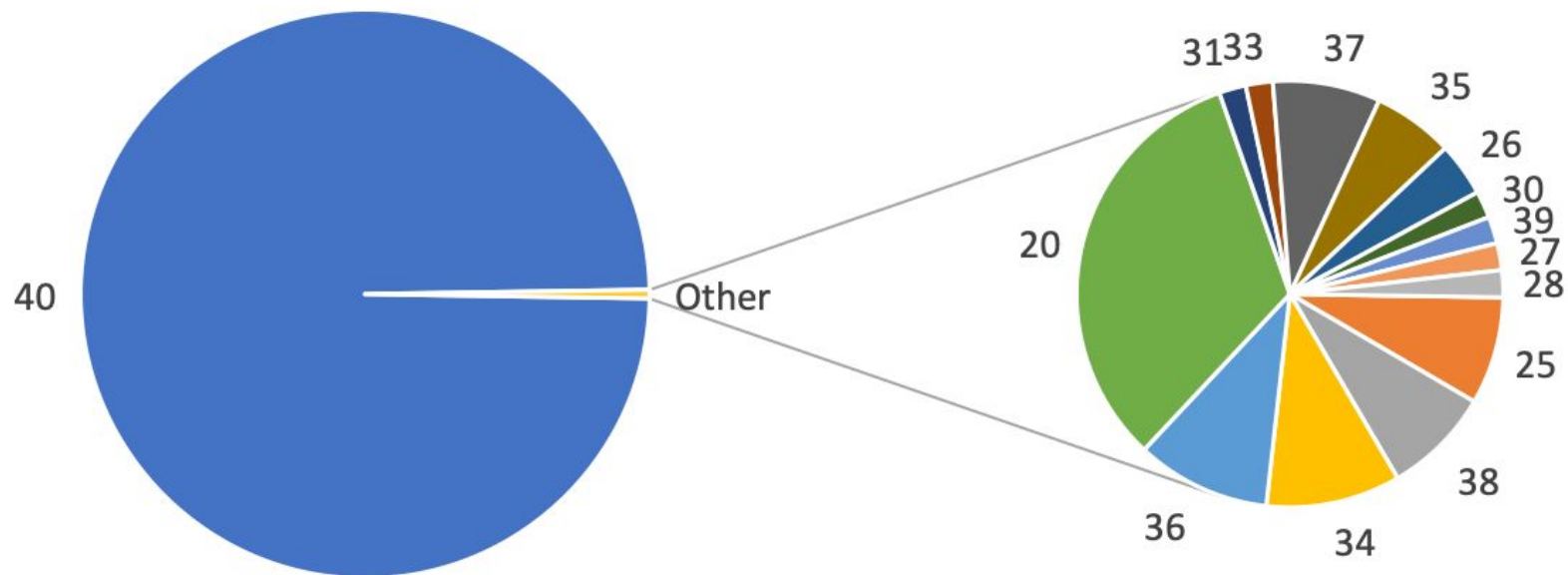
```
counter = 35 occurred 2 times
```

```
counter = 21 occurred 1 times
```

10,000 runs - 2 threads, 20 iterations



10,000 runs - 2 threads, 20 iterations



Exercise 1C

Change the program to use `NUM_THREADS=5` and `NUM_ITER=1000000`, and run it a few times.

What range of answers do you see? How often does it produce the answer that a naive programmer might expect?

Question 1C

— — —

```
-bash-4.2$ ./race
counter = 1217153
-bash-4.2$ ./race
counter = 1092655
-bash-4.2$ ./race
counter = 1194355
-bash-4.2$ ./race
counter = 1170149
-bash-4.2$ ./race
counter = 1141536
-bash-4.2$ ./race
counter = 1131291
-bash-4.2$ ./race
counter = 1183273
-bash-4.2$ ./race
counter = 1118640
-bash-4.2$ ./race
counter = 1261233
-bash-4.2$ ./race
counter = 1246367
-bash-4.2$ ./race
counter = 1177607
-bash-4.2$ ./race
counter = 1191164
-bash-4.2$ ./race
counter = 1145001
```

Exercise 1D

Declare and use a **mutex** to protect the increment of counter, then re-run the program.

Exercise 1D

```
mutex m;
```

```
int counter = 0;
```

```
void child() {  
    for (int i = 0; i < NUM_ITER; i++) {  
        m.lock();  
        counter++;  
        m.unlock();  
    }  
}
```

[illegible]

Exercise 2: Mutual exclusion

Assume:

atomic load and store

```
active[0] = false;
active[1] = false;
turn = 0;
```

Does this provide mutual
exclusion for the
critical section?

```
while(1) {
    active[0] = true;

    while (turn != 0) {
        while (active[1]) {}
        turn = 0;
    }
    <CRITICAL SECTION>
    active[0] = false;
}
```

```
while(1) {
    active[1] = true;

    while (turn != 1) {
        while (active[0]) {}
        turn = 1;
    }
    <CRITICAL SECTION>
    active[1] = false;
}
```



```
active[0] = false;  
active[1] = true;  
turn = 0;
```

```
while(1) {  
    active[0] = true;  
  
    while (turn != 0) {  
        while (active[1]) {}  
        turn = 0;  
    }  
    <CRITICAL SECTION>  
    active[0] = false;  
}
```

```
while(1) {  
    active[1] = true;  
  
    while (turn != 1) {  
        while (active[0]) {}  
        turn = 1;  
    }  
    <CRITICAL SECTION>  
    active[1] = false;  
}
```

```
active[0] = false;  
active[1] = true;  
turn = 0;
```

```
while(1) {  
    active[0] = true;  
  
    while (turn != 0) {  
        while (active[1]) {}  
        turn = 0;  
    }  
    <CRITICAL SECTION>  
    active[0] = false;  
}
```

```
while(1) {  
    active[1] = true;  
  
    while (turn != 1) {  
        while (active[0]) {}  
        turn = 1;  
    }  
    <CRITICAL SECTION>  
    active[1] = false;  
}
```



```
active[0] = true;  
active[1] = true;  
turn = 0;
```

```
while(1) {  
    active[0] = true;  
  
    while (turn != 0) {  
        while (active[1]) {}  
        turn = 0;  
    }  
    <CRITICAL SECTION>  
    active[0] = false;  
}
```

```
while(1) {  
    active[1] = true;  
  
    while (turn != 1) {  
        while (active[0]) {}  
        turn = 1;  
    }  
    <CRITICAL SECTION>  
    active[1] = false;  
}
```

```
active[0] = true;  
active[1] = true;  
turn = 0;
```

```
while(1) {  
    active[0] = true;  
  
    while (turn != 0) {  
        while (active[1]) {}  
        turn = 0;  
    } // SKIP  
    <CRITICAL SECTION>  
    active[0] = false;  
}
```

```
while(1) {  
    active[1] = true;  
  
    while (turn != 1) {  
        while (active[0]) {}  
        turn = 1;  
    }  
    <CRITICAL SECTION>  
    active[1] = false;  
}
```

```
active[0] = true;
active[1] = true;
turn = 1;
```

```
while(1) {
    active[0] = true;

    while (turn != 0) {
        while (active[1]) {}
        turn = 0;
    } // SKIPPED
    <CRITICAL SECTION>
    active[0] = false;
}
```



```
while(1) {
    active[1] = true;

    while (turn != 1) {
        while (active[0]) {}
        turn = 1;
    }
    <CRITICAL SECTION>
    active[1] = false;
}
```

```
active[0] = true;  
active[1] = true;  
turn = 1;
```

```
while(1) {  
    active[0] = true;  
  
    while (turn != 0) {  
        while (active[1]) {}  
        turn = 0;  
    } // SKIPPED  
    <CRITICAL SECTION>  
    active[0] = false;  
}
```

```
while(1) {  
    active[1] = true;  
  
    while (turn != 1) {  
        while (active[0]) {}  
        turn = 1;  
    }  
    <CRITICAL SECTION>  
    active[1] = false;  
}
```

```
active[0] = true;  
active[1] = true;  
turn = 1;
```

```
while(1) {  
    active[0] = true;  
  
    while (turn != 0) {  
        while (active[1]) {}  
        turn = 0;  
    } // SKIPPED  
    <CRITICAL SECTION>  
    active[0] = false;  
}
```

```
while(1) {  
    active[1] = true;  
  
    while (turn != 1) {  
        while (active[0]) {}  
        turn = 1;  
    }  
    <CRITICAL SECTION>  
    active[1] = false;  
}
```

```
active[0] = true;  
active[1] = true;  
turn = 1;
```

```
while(1) {  
    active[0] = true;  
  
    while (turn != 0) {  
        while (active[1]) {}  
        turn = 0;  
    } // SKIPPED  
    <CRITICAL SECTION>  
    active[0] = false;  
}
```

```
while(1) {  
    active[1] = true;  
  
    while (turn != 1) {  
        while (active[0]) {}  
        turn = 1;  
    }  
    <CRITICAL SECTION>  
    active[1] = false;  
}
```


Project 1

Project 1

— — —

- Due June 2 (Monday, ~2.5 weeks from now)
- 5% of your overall grade
- Implement a pizza delivery system
- Practice writing a concurrent program using monitors

What you are given

Thread library (libthread.o)

- thread
- mutex
- cv
- semaphore (don't use in project, just play around)

No source code; only the library object file and headers

Makefile

- Capable of compiling the project out-of-the-box, but nothing more
- Feel free to modify however you choose, as it (and other starter files) are not submitted. However, do not modify the contents of the pizza recipe (e.g., autotag.sh).
- Modify this line to list your *.cpp files:

Space-separated list of source files for the program

PIZZA_SOURCES := pizza.cpp

Project 1 Infrastructure: thread.h

- **thread::thread(thread_startfunc_t func, uintptr_t arg)**
- Spawns a thread
 - *The lifetime of the thread is **not** tied to the lifetime of the thread object*
- **thread_startfunc_t**
using thread_startfunc_t = void (*)(uintptr_t);
 - Just a type defined in thread.h to be “function which takes one uintptr_t as its argument and returns void”
- **func:** function pointer for the thread
 - Spawned thread executes this code

Project 1 Infrastructure: thread.h

- **thread::thread(thread_startfunc_t func, uintptr_t arg)**
- Spawns a thread
 - *The lifetime of the thread is **not** tied to the lifetime of the thread object*
- **uintptr_t**
 - An unsigned integer large enough to store a pointer
- **arg:** data to pass to the thread's function

Project 1 Infrastructure: thread.h

thread::join()

- Blocks the calling thread until the stream of execution associated with this thread object terminates
- If the stream of execution has already finished, join() is just a no-op

Project 1 Infrastructure: cpu.h

- `static void cpu::boot(thread_startfunc_t func, uintptr_t arg, unsigned int deterministic);`
- Spawns the main thread
- Must be called before any other infrastructure functions
- **func:** function pointer for main thread
- **arg:** data to pass to main thread
- **deterministic:**
 - 0 for nondeterministic (possibly different results for each run)
 - >0 for deterministic (same results for each run with the same value; possibly different results with different values)

What's wrong with this code?

```
int num = 0;
mutex m;

void func(uintptr_t) { cout << "num = " << num << endl; }

int main() {
    m.lock();      <-- Calling infrastructure function
    num = 42;      before cpu::boot()
    m.unlock();
    cpu::boot(func, 0, 0);
}
```

What's wrong with this code? (Fixed)

```
int num = 0;
mutex m;

void func(uintptr_t) { cout << "num = " << num << endl; }

int main() {
    // m.lock();
    num = 42;
    // m.unlock();
    cpu::boot(func, 0, 0);
}
```

Passing numbers to Threads

```
void func(uintptr_t my_num) {  
    cout << my_num << endl;  
}  
  
// Called from cpu::boot()  
void main_thread(uintptr_t) {  
    // Spawn 4 threads and give each one a different num  
    for (uintptr_t i = 0; i < 4; ++i) {  
        auto num = i * 42;  
        thread t(func, num);  
    }  
}
```

Pass Pointers to Threads

```
void func(uintptr_t p) {
    auto* my_num = reinterpret_cast<int*>(p); // my_num has type int*
    cout << *my_num << endl;
    delete my_num;
}

// Called from cpu::boot()
void main_thread(uintptr_t) {
    // Spawn 4 threads and give each one a different num
    for (auto i = 0; i < 4; ++i) {
        auto* num = new int(i * 42);
        thread t(func, reinterpret_cast<uintptr_t>(num));
    }
}
```

What's wrong with this code?

```
void func(uintptr_t p) {
    auto* my_num = reinterpret_cast<int*>(p); // my_num has type int*
    cout << *my_num << endl;
}

// Called from cpu::boot()
void main_thread(uintptr_t) {
    // Spawn 4 threads and give each one a different num
    for (auto i = 0; i < 4; ++i) {
        auto num = i * 42;
        thread t(func, reinterpret_cast<uintptr_t>(&num));
    }
}
```

After for-loop iteration ends, num goes out of scope

Fetch and return! (x4)

```
void func(uintptr_t p) {
    auto* my_num = reinterpret_cast<int*>(p);
    cout << *my_num << endl;
    delete my_num;
}

void main_thread(uintptr_t) {
    std::deque<thread> threads;
    for (auto num = 0; i < 4; ++i) {
        threads.emplace_back(func,
            reinterpret_cast<uintptr_t>(new int(num))); }
    for (auto& t : threads) { t.join(); }
    cout << "All done! (x4)" << endl;
}
```

Example Walkthrough

Driver 0

0 0

Driver 1

0 0

Customer 0

0 4
5 0

Customer 1

7 3
1 0

Driver 0

0 0 ready

Driver 1

0 0

Customer 0

0 4
5 0

Customer 1

7 3
1 0

Driver 0

0 0 ready

Customer 0

0 4
5 0

Driver 1

0 0 ready

Customer 1

7 3
1 0

Driver 0

0 0 ready

Driver 1

0 0 ready

Customer 0

0 4 requesting

5 0

Customer 1

7 3

1 0

Driver 0

0 0 ready

Driver 1

0 0 ready

Customer 0

0 4 requesting

5 0

Customer 1

7 3 requesting

1 0

Driver 0

0 0 ready

match

Customer 0

0 4 requesting
5 0

Driver 1

0 0 ready

Customer 1

7 3 requesting
1 0

A ready driver d and ready customer c match
iff
 c is the closest customer to d and d is the closest
driver to c

Driver 0

0	0	driving
---	---	----------------

match

Customer 0

0	4	requesting
5	0	

Driver 1

0	0	ready
---	---	--------------

Customer 1

7	3	requesting
1	0	

Driver 0

0	0	driving
---	---	----------------

match

Customer 0

0	4	requesting
5	0	

Driver 1

0	0	ready
---	---	--------------

match

Customer 1

7	3	requesting
1	0	

Driver 0

0	0	driving
---	---	----------------

match

Customer 0

0	4	requesting
5	0	

Driver 1

0	0	driving
---	---	----------------

match

Customer 1

7	3	requesting
1	0	

Driver 0

0 4 arrived

match

Customer 0

0 4 pays
5 0

Driver 1

0 0 driving

match

Customer 1

7 3 requesting
1 0

Driver 0

0 4 arrived

match

Customer 0

0 4 pays
5 0

Driver 1

7 3 arrived

match

Customer 1

7 3 pays
1 0

Driver 0

0 4 arrived

match

Customer 0

0 4 **pays**

5 0

Driver 1

7 3 arrived

Customer 1

7 3 completed

1 0 **requesting**

Driver 0

0 4 arrived

Customer 0

0 4 completed

5 0

Driver 1

7 3 ready

Customer 1

7 3 completed

1 0 requesting

Driver 0

0 4 arrived

Customer 0

0 4 completed

5 0

Driver 1

7 3 ready

match

Customer 1

7 3 completed

1 0 requesting

Driver 0

0 4 arrived

Customer 0

0 4 completed

5 0

Driver 1

7 3 driving

match

Customer 1

7 3 completed

1 0 requesting

Driver 0

0 4 ready

Customer 0

0 4 completed

5 0

Driver 1

7 3 driving

match

Customer 1

7 3 completed

1 0 requesting



Driver 0

0 4 ready

Customer 0

0 4 completed

5 0 requesting

Driver 1

7 3 driving

match

Customer 1

7 3 completed

1 0 requesting



Driver 0

0 4 ready

match

Customer 0

0 4 completed
5 0 requesting

Driver 1

7 3 driving

match

Customer 1

7 3 completed
1 0 requesting

Driver 0

0 4 driving

match

Customer 0

0 4 completed
5 0 requesting

Driver 1

7 3 driving

match

Customer 1

7 3 completed
1 0 requesting

Driver 0

5 0 arrived

match

Customer 0

0 4 completed
5 0 pays

Driver 1

7 3 driving

match

Customer 1

7 3 completed
1 0 requesting

Driver 0

5 0 ready

Customer 0

0 4 completed

5 0 completed

Driver 1

1 0 arrived

match

Customer 1

7 3 completed

1 0 pays



Driver 0

5 0 ready

Customer 0

0 4 completed

5 0 completed

Driver 1

1 0 ready

Customer 1

7 3 completed

1 0 completed

Coding Tips for P1

[Link to Slides](#)

- Function Pointers
- Makefile

C++ Threads

std::thread (C++11) vs Project 1 Thread

- std::thread is the C++ STL's threading API

```
template <class Callable, class ...Args>
    explicit thread(Callable&&, Args&&...);
```
- Let's unpack this! (pun intended)
 - Callable is any type that is callable, meaning that it supports operator() and can be called via "f()" syntax
 - Args is a pack of parameters of **any** type
- You can initialize a thread as long as the Callable object can be invoked with the Args parameters

What's wrong with this code?

```
void func(int my_num) { cout << my_num << endl; }

int main() {
    // Spawn 4 threads and give each one a different num
    for (auto i = 0; i < 4; ++i) {
        auto num = i * 42;
        // note the use of reinterpret_cast
        std::thread t(func, num);
    }
}
```

- C++ threads calls `terminate()` if it is destroyed in a “joinable” state
- C++ thread is “joinable” if `joinable()` returns true

What's wrong with this code?

```
void func(int my_num) { cout << my_num << endl; }

int main() {
    // Spawn 4 threads and give each one a different num
    for (auto i = 0; i < 4; ++i) {
        auto num = i * 42;
        // note the use of reinterpret_cast
        std::thread t(func, num);
        t.join();
    }
}
```

- No concurrency now!
- Joining right after creating the thread makes it serial

Correct Solution

```
void func(int my_num) { cout << my_num << endl; }

int main() {
    std::vector<std::thread> threads;
    // Spawn 4 threads and give each one a different num
    for (auto i = 0; i < 4; ++i) {
        auto num = i * 42;
        // note the use of reinterpret_cast
        threads.emplace_back(func, num);
    }
    for (auto &t : threads) {
        t.join();
    }
}
```

Even better: `std::jthread` (C++20)

- Automatically joins upon thread object destruction

```
void func(int my_num) { cout << my_num << endl; }

int main() {
    std::vector<std::jthread> threads;
    // Spawn 4 threads and give each one a different num
    for (auto i = 0; i < 4; ++i) {
        auto num = i * 42;
        // note the use of reinterpret_cast
        thread.emplace_back(func, num);
    }
}
```

Questions?